02 · Numerical methods

From mathematical equations to robust, testable algorithms applied to real engineering problems.

MODULE	STARTING POINT	FOCUS
02_numerical_methods	Class 17 · EX25	Engineering + Python + testing

Module goal

Learn not only to calculate a number, but also to justify the algorithm, estimate its error, verify its limits and explain its physical meaning.

1 · Learning roadmap

Six connected units, sequenced to develop mathematical judgement and implementation skills. No fixed dates or durations: we progress once each concept and its validations are mastered.

01	**Numerical algebra**	algebra/ Vectors, matrices, linear systems and numerical precision. Force allocation and actuator models.
02	**Root finding**	root_finding/ Bisection, Newton-Raphson, tolerances and convergence. Solving nonlinear engineering equations.
03	**Numerical differentiation**	differentiation/ Finite differences, time steps and numerical error. Estimate velocity and acceleration from data.
04	**Numerical integration**	integration/ Trapezoidal rule, Simpson, discretisation and comparison. Accumulate physical quantities from measurements or functions.
05	**Statistics and probability**	statistics/ Uncertainty, error propagation and Monte Carlo. Characterise measurements and quantify variability.
06	**Differential equations**	ode/ Euler, fourth-order Runge-Kutta, stability and step size. Simulate dynamic systems.

2 · What you will learn

Each unit combines mathematical concepts, Python tools and an engineering application with reproducible checks.

01 · NUMERICAL ALGEBRA

Represent data as arrays; perform vector and matrix operations; solve A·x = b; interpret residuals, conditioning and precision.

Python: NumPy · np.array · @ · np.linalg.solve · np.linalg.norm · np.linalg.cond

Engineering: EX25: force allocation for two actuators. Validate forces, residual and sensitivity.

02 · ROOT FINDING

Formulate f(x) = 0; distinguish brackets and initial guesses; set tolerances; detect failure to converge.

Python: Functions · loops · bisection · Newton-Raphson · stopping criteria

Engineering: Find an equilibrium condition or a system operating point.

03 · NUMERICAL DIFFERENTIATION

Approximate derivatives with forward, backward and central differences; compare errors and noise.

Python: Arrays · slicing · np.diff · analytical derivative comparisons

Engineering: Recover velocity and acceleration from sampled position data.

04 · NUMERICAL INTEGRATION

Approximate areas and integrals using trapezoidal and Simpson rules; study resolution and convergence.

Python: Loops · NumPy · cumulative sums · integration functions

Engineering: Estimate distance, impulse or energy from functions or data.

05 · STATISTICS AND PROBABILITY

Summarise samples, quantify spread and propagate uncertainty; perform Monte Carlo simulations.

Python: NumPy · reproducible random generator · percentiles · plots

Engineering: Assess sensor uncertainty and propagate it into the result.

06 · DIFFERENTIAL EQUATIONS

Discretise dynamic equations; implement Euler and RK4; study stability and dependence on Δt.

Python: State functions · time integration · arrays · matplotlib

Engineering: Simulate motion or physical dynamics and compare with a reference.

3 · How we work

Our standard problem-solving process for every mission in this module.

01	**Physical problem**	Define the system, units, assumptions and engineering question.
02	**Mathematics**	Formulate equations, variables, conditions and expected outcomes.
03	**Algorithm**	Convert the model into implementable steps and stopping rules.
04	**Python**	Write clear, reusable functions with defined inputs and outputs.
05	**Validation**	Compare answers with known solutions or hand calculations.
06	**Interpretation**	Analyse error, sensitivity and physical meaning of the results.
07	**Testing**	Check normal, edge and error cases with automated tests.

4 · Class materials and deliverables

For every class/exercise: six PDFs (mission, reference guide and theory guide, each in Spanish and English). At the start of each new section: a dedicated bilingual roadmap.

5 · Project folder structure

```
Python_Engineering/
  02_numerical_methods/
    roadmap/
      Python_Engineering_Module_02_Roadmap_ES.pdf
      Python_Engineering_Module_02_Roadmap_EN.pdf
    algebra/  <- start here
      class_17_vectors_and_linear_systems/
    root_finding/
    differentiation/
    integration/
    statistics/
    ode/
```

> **Completion criteria for each unit**
>
> Understand the model and functions; solve an engineering mission; validate against references; explain accuracy and limitations; pass the relevant tests.

Class and exercise numbering carries over from the previous module. Class 17 / EX25 is the first exercise here; subsequent missions will be assigned as we progress.